CGE-P LAB NOTES

Lab 3.4: Integrating PaC with Terraform via Conftest (AWS)

You wrote five Rego policies in Lab 3.3 against GCP fixtures. This lab does two things. It runs them against an AWS Terraform plan via Conftest, and it forces you to add AWS variants, because rules that hardcode `google_storage_bucket` match nothing in an AWS plan.

The library survives the cloud change because the control IDs do. The rules do not.

For reading. Code lines wrap to fit the page; copy code from docs/03_04_integrating_pac_with_terraform.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/03_04_integrating_pac_with_terraform.md` in your clone, not from the PDF.

Learning objectives

- Wire Conftest into the plan workflow as a fail-closed gate.
- Add AWS variants for SC-8, SC-28, AC-3, AU-9, and CM-6, preserving control IDs.
- Understand why some controls cannot be fully asserted at plan time, and what to do about it.
- Demonstrate a blocked merge by feeding a deliberately broken plan to the gate.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Lab 2.3 workspace on disk. Its plan is the input, and it contains a KMS key and two bucket policies the rules below depend on.
- Lab 3.3 policy library carried forward. Five rules, five controls.
- Conftest `>= 0.50`.
- OPA `>= 0.60`, for `opa test` and the mutation script.

Estimated time & cost

- 60 minutes.
- Free. No AWS resources beyond Lab 2.3.

Step-by-step walkthrough

Step 1 Carry the library forward and re-test

```
cp -r ../lab-3-3/policies ./policies
cp -r ../lab-3-3/scripts  ./scripts
opa test -v policies/
bash scripts/mutation-test.sh
```

Sanity check before extending. If the mutation script reports a survivor now, fix it now; adding five more rules on top of an untrusted base compounds the problem.

Step 2 Generate the AWS plan

```
cd ../lab-2-3
export AWS_PROFILE=cgep
terraform init
terraform plan -out=tfplan
terraform show -json tfplan > plan.json
```

Step 3 The cross-cloud lesson

Run the GCP policies against the AWS plan:

```
conftest test --policy policies --namespace compliance.sc28 plan.json
conftest test --policy policies --namespace compliance.ac3  plan.json
conftest test --policy policies --namespace compliance.au9  plan.json
```

They pass, with zero coverage. There are no `google_storage_bucket` resources here, so the rules match nothing and deny nothing.

Sit with that for a moment, because it is the same failure mode as an untested policy. A rule that matches nothing is indistinguishable from a rule that found nothing wrong. Conftest reports `0 failures` for both. The only difference is whether your infrastructure is compliant or your policy is inert.

That is why Lab 3.3 ends with mutation testing, and it is why the AWS variants below are separate files rather than a generalized rule. Two readable rules beat one clever rule whose coverage you cannot eyeball.

Step 4 SC-28 for AWS, now requiring a CMK

Asking whether *any* encryption configuration exists is the weaker question. Lab 2.3 uses a customer-managed key and the capstone requires one, so this rule asks whether the right kind is in place.

```
# policies/sc28_encryption_aws.rego
# METADATA
# title: SC-28 - Encryption at Rest (AWS S3, customer-managed key)
# description: "Every aws_s3_bucket must have a server-side encryption configuration using aws:kms
with a customer-managed key."
# custom:
#   control_id: SC-28
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add aws_s3_bucket_server_side_encryption_configuration with sse_algorithm =
\"aws:kms\", kms_master_key_id referencing your aws_kms_key, and bucket_key_enabled = true."
package compliance.sc28_aws

import rego.v1

deny contains msg if {
    some bucket in bucket_addresses
```

```
    not has_encryption(bucket)
    msg := sprintf(
        "[SC-28] %s: no aws_s3_bucket_server_side_encryption_configuration references this bucket.
Remediation: add one.",
        [bucket],
    )
}

deny contains msg if {
    some bucket in bucket_addresses
    has_encryption(bucket)
    not has_kms_encryption(bucket)
    msg := sprintf(
        "[SC-28] %s: encrypted with SSE-S3 (AES256), not a customer-managed key. Remediation:
sse_algorithm = \"aws:kms\" with kms_master_key_id.",
        [bucket],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

sse_configs contains r if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_server_side_encryption_configuration"
}

has_encryption(bucket_addr) if {
    some r in sse_configs
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

has_kms_encryption(bucket_addr) if {
    some r in sse_configs
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
    r.expressions.rule[_].apply_server_side_encryption_by_default[_].sse_algorithm.constant_value ==
"aws:kms"
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.arn", [addr])
```

Why match by reference, not by value. At plan time the bucket name is "(known after apply)" because the `random_id` suffix has not been generated. Both `aws_s3_bucket.values.bucket` and the encryption resource's `values.bucket` are `null` in the JSON. Use `configuration.root_module.resources[].expressions.bucket.references`, which holds strings like `"aws_s3_bucket.primary.id"` that Terraform resolves at apply.

Two separate `deny` rules rather than one compound condition. A missing configuration and a weak configuration are different findings with different remediations, and merging them produces a message that is wrong half the time.

Step 5 SC-8 for AWS, and the limits of plan-time policy

This is the hardest of the five to assert, and the reason is worth understanding.

A bucket policy is a JSON string. In Lab 2.3 it is produced by an `aws_iam_policy_document` data source whose inputs include bucket ARNs, which depend on names, which depend on `random_id`. **At plan time the rendered policy JSON is unknown.** You cannot parse what Terraform has not computed.

So the rule works in two tiers, and says so:

```
# policies/sc8_tls_required_aws.rego
# METADATA
# title: SC-8 - Transmission Confidentiality (S3 TLS enforcement)
# description: "Every aws_s3_bucket must have a bucket policy, and that policy must deny requests
where aws:SecureTransport is false."
# custom:
#   control_id: SC-8
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add an aws_s3_bucket_policy with a Deny statement on s3:* when Bool
aws:SecureTransport = false."
#   plan_time_limitation: "Rendered policy JSON is unknown at plan time when bucket ARNs are computed.
Tier 1 (a policy exists) always evaluates; tier 2 (it denies non-TLS) evaluates only when the JSON is
known. Post-apply verification closes the gap."
package compliance.sc8_aws

import rego.v1

# Tier 1: structural. Always evaluable. A bucket with no policy at all cannot
# be enforcing TLS.
deny contains msg if {
    some bucket in bucket_addresses
    not has_policy(bucket)
    msg := sprintf(
        "[SC-8] %s: no aws_s3_bucket_policy references this bucket, so non-TLS requests are permitted.
Remediation: add a policy denying aws:SecureTransport = false.",
        [bucket],
    )
}

# Tier 2: semantic. Only fires when the rendered JSON is known, which happens
# for hardcoded bucket names or on a re-plan against existing state.
deny contains msg if {
    some r in input.planned_values.root_module.resources
    r.type == "aws_s3_bucket_policy"
    is_string(r.values.policy)
    doc := json.unmarshal(r.values.policy)
    not denies_insecure_transport(doc)
    msg := sprintf(
```

```
        "[SC-8] %s: bucket policy is present but contains no Deny on aws:SecureTransport = false.",
        [r.address],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

has_policy(bucket_addr) if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_policy"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

denies_insecure_transport(doc) if {
    some stmt in doc.Statement
    stmt.Effect == "Deny"
    stmt.Condition.Bool["aws:SecureTransport"] == "false"
}

denies_insecure_transport(doc) if {
    some stmt in doc.Statement
    stmt.Effect == "Deny"
    some v in stmt.Condition.Bool["aws:SecureTransport"]
    v == "false"
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

Two `denies_insecure_transport` definitions because IAM condition values may be a bare string or an array, and both are valid JSON policy. Rego's multiple-definition semantics handle that cleanly: either matches.

Teach the limitation explicitly. A student who believes tier 2 always runs will trust a gate that is only doing tier 1. The honest statement is: at plan time we prove a policy exists; that it says the right thing is verified after apply, by the `aws s3api get-bucket-policy` check in Lab 2.3's verification section, and continuously by Security Hub in Lab 5.2. Policy-as-code is one layer of three, and pretending otherwise is how people end up with confident dashboards and open buckets.

Step 6 AC-3, AU-9, and CM-6 for AWS

```
# policies/ac3_no_public_aws.rego
# METADATA
# title: AC-3 - Access Enforcement (AWS S3 public access block)
# description: "Every aws_s3_bucket must have an aws_s3_bucket_public_access_block with all four flags
true."
```

```
# custom:
#   control_id: AC-3
#   framework: nist-800-53-rev5
#   severity: critical
#   remediation: "Add aws_s3_bucket_public_access_block with all four flags set to true. Three is not
enough."
package compliance.ac3_aws

import rego.v1

deny contains msg if {
    some bucket in bucket_addresses
    not has_complete_pab(bucket)
    msg := sprintf(
        "[AC-3] %s: missing or incomplete aws_s3_bucket_public_access_block. All four flags must be
true.",
        [bucket],
    )
}

bucket_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket"
    addr := sprintf("aws_s3_bucket.%s", [r.name])
}

has_complete_pab(bucket_addr) if {
    pab := pab_for(bucket_addr)
    v := pab_planned_values(pab)
    v.block_public_acls == true
    v.block_public_policy == true
    v.ignore_public_acls == true
    v.restrict_public_buckets == true
}

pab_for(bucket_addr) := addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_public_access_block"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
    addr := sprintf("aws_s3_bucket_public_access_block.%s", [r.name])
}

pab_planned_values(addr) := values if {
    some r in input.planned_values.root_module.resources
    r.address == addr
    values := r.values
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

```
# policies/au9_log_immutability_aws.rego
# METADATA
# title: AU-9 - Protection of Audit Information (S3 log bucket versioning)
# description: "A bucket named as an aws_s3_bucket_logging target must itself have versioning
enabled."
# custom:
#   control_id: AU-9
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add aws_s3_bucket_versioning for the log bucket with status = \"Enabled\"."
package compliance.au9_aws

import rego.v1

deny contains msg if {
    some target in log_target_addresses
    not versioned(target)
    msg := sprintf(
        "[AU-9] %s: used as an access-log target but has no aws_s3_bucket_versioning. Audit records
can be silently overwritten.",
        [target],
    )
}

# The target_bucket expression references the log bucket's address.
log_target_addresses contains addr if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_logging"
    some ref in r.expressions.target_bucket.references
    addr := trim_suffix(trim_suffix(ref, ".id"), ".bucket")
    startswith(addr, "aws_s3_bucket.")
}

versioned(bucket_addr) if {
    some r in input.configuration.root_module.resources
    r.type == "aws_s3_bucket_versioning"
    some ref in r.expressions.bucket.references
    references_bucket(ref, bucket_addr)
}

references_bucket(ref, addr) if ref == addr
references_bucket(ref, addr) if ref == sprintf("%s.id", [addr])
references_bucket(ref, addr) if ref == sprintf("%s.bucket", [addr])
```

Run that one against a plan where only the primary bucket is versioned. It fires. That rule is the mechanical form of a mistake almost everybody makes once: an architecture that promises versioning on both buckets, and code that versions the one holding data. A policy suite catches it on the first pull request, which is the argument for policy-as-code stated more persuasively than any slide.

```
# policies/cm6_required_tags_aws.rego
# METADATA
# title: CM-6 - Configuration Settings (AWS required tags)
# description: "Every taggable resource must carry the four required tags."
```

```
# custom:
#   control_id: CM-6
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add the four required tags, or set them once via provider default_tags."
package compliance.cm6_aws

import rego.v1

required := {"Project", "Environment", "ManagedBy", "ComplianceScope"}

taggable_type(t) if t == "aws_s3_bucket"
taggable_type(t) if t == "aws_kms_key"
taggable_type(t) if t == "aws_dynamodb_table"
taggable_type(t) if t == "aws_lambda_function"
taggable_type(t) if t == "aws_cloudtrail"

deny contains msg if {
    some resource in all_resources
    taggable_type(resource.type)
    missing := required - tag_keys(resource)
    count(missing) > 0
    msg := sprintf(
        "[CM-6] %s: missing required tags %v. Remediation: add them, or use provider default_tags.",
        [resource.address, sort_array(missing)],
    )
}

all_resources contains r if { some r in input.planned_values.root_module.resources }

all_resources contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
}

# With provider default_tags, the merged set lives in tags_all.
tag_keys(resource) := keys if {
    resource.values.tags_all
    keys := {k | resource.values.tags_all[k]}
}

tag_keys(resource) := keys if {
    not resource.values.tags_all
    resource.values.tags
    keys := {k | resource.values.tags[k]}
}

tag_keys(resource) := set() if {
    not resource.values.tags_all
    not resource.values.tags
}

sort_array(s) := sort([x | some x in s])
```

Note `aws_kms_key` in the taggable list. Lab 2.3 creates one, and an untagged key is a resource outside your boundary enumeration.

Step 7 Run the gate against the compliant plan

```
for ns in sc8_aws sc28_aws ac3_aws au9_aws cm6_aws; do
  echo "=== compliance.$ns ==="
  conftest test --policy policies --namespace "compliance.$ns" plan.json
done
```

Expected: five namespaces, zero failures. Lab 2.3 now has full AWS coverage from your library.

Step 8 Break it three ways

One broken plan per control class, because a gate you have only watched pass is a gate you are guessing about.

```
mkdir -p broken && cp ../lab-2-3/*.tf broken/
```

1. **SC-28**: change `sse_algorithm` to `"AES256"` and drop `kms_master_key_id`.
2. **SC-8**: delete `aws_s3_bucket_policy.primary`.
3. **AU-9**: delete `aws_s3_bucket_versioning.log`.

Then:

```
( cd broken && terraform init && terraform plan -out=tfplan && terraform show -json tfplan >
plan.json )

for ns in sc8_aws sc28_aws au9_aws; do
  conftest test --policy policies --namespace "compliance.$ns" broken/plan.json
done
```

```
FAIL - broken/plan.json - compliance.sc28_aws - [SC-28] aws_s3_bucket.primary: encrypted with SSE-S3
(AES256), not a customer-managed key. ...
FAIL - broken/plan.json - compliance.sc8_aws  - [SC-8] aws_s3_bucket.primary: no aws_s3_bucket_policy
references this bucket, ...
FAIL - broken/plan.json - compliance.au9_aws  - [AU-9] aws_s3_bucket.log: used as an access-log target
but has no aws_s3_bucket_versioning. ...
```

Non-zero exit. Each message names the resource, the control, and the fix.

Step 9 The wrapper script

```
#!/usr/bin/env bash
# scripts/policy-gate.sh
# Fail-closed Conftest gate over a Terraform plan.
#
# Usage:
```

```
#   policy-gate.sh --workspace <dir>   # runs `terraform show -json tfplan` there
#   policy-gate.sh --plan <plan.json>  # uses an existing plan JSON
set -euo pipefail

POLICY_DIR="policies"
WORKSPACE=""
PLAN=""
EVIDENCE_DIR="evidence/lab-3-4"

NAMESPACES=(
  compliance.sc8_aws
  compliance.sc28_aws
  compliance.ac3_aws
  compliance.au9_aws
  compliance.cm6_aws
)

while [[ $# -gt 0 ]]; do
  case "$1" in
    --workspace) WORKSPACE="$2"; shift 2 ;;
    --plan)      PLAN="$2";      shift 2 ;;
    --policy)    POLICY_DIR="$2"; shift 2 ;;
    --evidence)  EVIDENCE_DIR="$2"; shift 2 ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

if [[ -z "$PLAN" ]]; then
  [[ -z "$WORKSPACE" ]] && { echo "Usage: $0 --workspace <dir> | --plan <plan.json>" >&2; exit 2; }
  ( cd "$WORKSPACE" && terraform show -json tfplan > plan.json )
  PLAN="$WORKSPACE/plan.json"
fi

mkdir -p "$EVIDENCE_DIR"
RESULTS="$EVIDENCE_DIR/conftest-results.json"
EXIT=0

{
  echo "["
  FIRST=1
  for ns in "${NAMESPACES[@]}"; do
    [[ $FIRST -eq 1 ]] && FIRST=0 || printf ","
    OUT=$(conftest test --policy "$POLICY_DIR" --namespace "$ns" --output=json "$PLAN" || true)
    echo "${OUT:-[]}"
    if ! echo "${OUT:-[]}" | jq -e 'all(.[]; (.failures // []) | length == 0)' >/dev/null 2>&1; then
      EXIT=1
    fi
  done
  echo "]"
} > "$RESULTS"

# Fail closed if the gate produced nothing. An empty result set is not a
# pass: a rule that matches nothing and a rule that found nothing wrong are
# indistinguishable in the output. This is the check that makes the gate a
# gate rather than a decoration.
if ! jq -e 'flatten | length > 0' "$RESULTS" >/dev/null 2>&1; then
```

```
  echo "policy-gate: FAIL (no policy results produced; gate did not run)" >&2
  exit 2
fi

if [[ $EXIT -eq 0 ]]; then
  echo "policy-gate: PASS"
else
  echo "policy-gate: FAIL"
  jq -r 'flatten | .[] | (.failures // [])[] | .msg' "$RESULTS" >&2
fi
exit $EXIT
```

Three details worth naming:

- `jq` **instead of an inline** `python3` **heredoc.** Same job, one dependency you already have, and no indentation hazard inside YAML later.
- **The empty-results guard.** Without it the gate returns `PASS` when Conftest fails to load any policy at all, because zero failures out of zero tests is zero failures. That is the inert-policy failure mode from Step 3, caught by the harness.
- **Failure messages echoed to stderr.** A developer reading a red PR should not have to download an artifact to learn what broke.

Verification

A gate has three outcomes and they have to be told apart: pass, fail on real findings, and **fail closed** because it had nothing to evaluate. The third is the one worth building carefully, and it is also the one this lab used to test incorrectly.

```
bash scripts/verify.sh
```

Exit 2 is not proof of a working guard. The old test for fail-closed was

```
policy-gate.sh --policy /tmp/no-policies-here plan.json
```

which exits 2 because `plan.json` is not a recognized argument, the script taking `--plan`. It never loads a policy and never reads the plan. That is the same exit code the guard produces, so the guard could have been deleted outright and this test would still have passed.

It was checked: removing the empty-results guard makes an empty policy directory exit **0**, a gate reporting success while enforcing nothing. That is precisely the decoration this lab exists to prevent, and the test that was supposed to catch it could not.

So the script asserts the **message**, not just the status, and then makes the argument mistake on purpose to prove the two are distinguishable.

Expect the compliant fixture to pass, the degraded one to exit 1 citing SC-8, SC-28 and AU-9, and an empty policy directory to exit 2 saying the gate did not run.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 3.4.
#
# A gate has three outcomes and they must be told apart. Pass, fail on real
# findings, and fail closed because it had nothing to evaluate. The third is
# the one that gets faked: the guide used to test it with
#
#   policy-gate.sh --policy /tmp/no-policies-here plan.json
#
# which exits 2 because `plan.json` is not a recognized argument, before a
# single policy is loaded or the plan is read. Same exit code as the guard it
# claimed to be testing, so the guard could have been deleted entirely and
# that test would still have passed.
set -uo pipefail

WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0
pass() { printf '  PASS  %-10s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-10s %s\n' "$1" "$2" >&2; FAILED=1; }

EV=$(mktemp -d)
trap 'rm -rf "$EV"' EXIT
EMPTY=$(mktemp -d)

run() { # run POLICY_DIR PLAN -> sets RC and OUT
  OUT=$(bash scripts/policy-gate.sh --policy "$1" --plan "$2" --evidence "$EV" 2>&1)
  RC=$?
}

echo "=== a compliant plan passes ==="
run policies fixtures/plan-compliant.json
if [[ $RC -eq 0 && "$OUT" == *"policy-gate: PASS"* ]]; then
  pass "compliant" "exit 0 and PASS"
else
  fail "compliant" "expected exit 0 with PASS, got exit $RC"
fi

echo "=== a degraded plan fails, citing the right controls ==="
run policies fixtures/plan-degraded.json
if [[ $RC -eq 1 ]]; then
  pass "degraded" "exit 1"
else
  fail "degraded" "expected exit 1, got $RC"
fi

CONTROLS=$(jq -r '[.[][]?.failures[]?.msg] | .[] | capture("\\[(?<c>[A-Z]+-[0-9.]+)\\]").c' \
  "$EV/conftest-results.json" 2>/dev/null | sort -u | paste -sd, -)
if [[ "$CONTROLS" == "AU-9,SC-28,SC-8" ]]; then
  pass "degraded" "cites AU-9, SC-28, SC-8"
```

```
else
  fail "degraded" "expected AU-9,SC-28,SC-8; got '${CONTROLS:-(none)}'"
fi

echo "=== a gate with no policies fails CLOSED, for the stated reason ==="
run "$EMPTY" fixtures/plan-compliant.json
if [[ $RC -eq 2 ]]; then
  pass "fail-closed" "exit 2"
else
  fail "fail-closed" "expected exit 2, got $RC"
fi

# Exit 2 alone proves nothing here: an argument typo produces it too. The
# message is what distinguishes a gate that refused to run from a gate that
# was never invoked.
if [[ "$OUT" == *"gate did not run"* ]]; then
  pass "fail-closed" "reported that the gate did not run"
else
  fail "fail-closed" "exit 2 for the wrong reason. Output was: $(head -1 <<<"$OUT")"
fi

# And prove the distinction is real, by making the mistake on purpose.
BAD=$(bash scripts/policy-gate.sh --policy policies fixtures/plan-compliant.json 2>&1)
BADRC=$?
if [[ $BADRC -eq 2 && "$BAD" == *"Unknown arg"* ]]; then
  pass "fail-closed" "a bad argument also exits 2, which is why the message is checked"
else
  fail "fail-closed" "expected a positional plan to be rejected with exit 2"
fi

rmdir "$EMPTY" 2>/dev/null
echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: the gate passes, fails, and fails closed, each for its own reason."
else
  echo "NOT VERIFIED: the gate does not behave as described. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-3-4"
mkdir -p "$EVIDENCE"

# Both runs, kept as separate artifacts. The gate always writes
# conftest-results.json, so each run is renamed before the next overwrites it.
bash scripts/policy-gate.sh --plan fixtures/plan-compliant.json --evidence "$EVIDENCE"
mv "$EVIDENCE/conftest-results.json" "$EVIDENCE/conftest-pass.json"

# The degraded run exits 1 by design, so it needs `|| true` or the mv is
# skipped and you lose the more interesting of the two files.
bash scripts/policy-gate.sh --plan fixtures/plan-degraded.json --evidence "$EVIDENCE" || true
```

```
mv "$EVIDENCE/conftest-results.json" "$EVIDENCE/conftest-fail.json"

# The three outcomes, each proven for its own reason.
bash scripts/verify.sh 2>&1 | tee "$EVIDENCE/inert-gate-test.txt"
```

Portfolio submission checklist

- [ ] `policies/` holds GCP and AWS variants: ten files, six control IDs.
- [ ] `scripts/policy-gate.sh` committed and executable.
- [ ] `evidence/lab-3-4/conftest-pass.json`
- [ ] `evidence/lab-3-4/conftest-fail.json`
- [ ] `evidence/lab-3-4/inert-gate-test.txt`, showing the gate exits non-zero with no policies present.
- [ ] `policies/README.md` noting which file targets which cloud, and the SC-8 plan-time limitation.

Troubleshooting

- **`policies: no such file or directory`.** `--policy` resolves relative to your shell. Pass an absolute or canonically-relative path.
- **`no policies matched`.** The `package` declaration must equal the `--namespace` string exactly.
- **A passing fixture fires anyway.** Module-wrapped resources live under `child_modules[]`.
- **Bucket name comparisons return undefined.** Plan-time IDs are unknown. Match by reference in `configuration...expressions.<arg>.references`.
- **SC-8 tier 2 never fires.** Expected. The rendered policy JSON is unknown when bucket ARNs are computed. Tier 1 still runs. Verify the semantics post-apply.
- **`jq: error: Cannot iterate over null`** in the gate. Conftest emits `null` rather than `[]` for a namespace with no results. The `// []` fallbacks handle it; keep them if you refactor.

Cleanup

Local. Nothing in the cloud beyond Lab 2.3.

These policies already detect capstone gaps. The starter's `GAPS.md` lists eight, and three of the AWS variants you just wrote catch one each, against `aws_s3_bucket.uploads` in the starter's own Terraform:

Policy	Catches
`sc28_encryption_aws`	GAP-01, SSE-S3 rather than a customer CMK
`sc8_tls_required_aws`	GAP-03, no `aws:SecureTransport` deny
`au9_log_immutability_aws`	GAP-04, no versioning

The capstone asks for **at least five** policies detecting the most material gaps, and requires that the grader can re-introduce a gap and watch your suite fail closed. Three of the five already exist. Point them at the starter's plan and see what happens before you write more.

How this feeds the capstone

`scripts/policy-gate.sh` is the exact script CI calls in Lab 4.3. The capstone's workflow shells out with `--workspace ./terraform`, and the plan is checked before apply.

Two things to carry forward. The empty-results guard is what makes the capstone's "we re-introduce a gap and confirm the gate fires" check survivable, because a gate that silently loads no policies passes that test in the worst possible way. And the SC-8 two-tier pattern is the honest shape for any control whose enforcement lives in a computed string: assert what you can at plan time, verify the rest after apply, and write down which is which.

Revision history

These notes

- Two AWS variants added: SC-8 (a bucket policy exists and denies non-TLS) and AU-9 (the access-log target is versioned).
- SC-28's AWS variant now distinguishes missing encryption from weak encryption, and requires a customer-managed key rather than any configuration.
- `policy-gate.sh` gained an empty-results guard, so a gate that loads no policies exits non-zero instead of reporting success.
- Gate failure messages are echoed to stderr rather than only written to the evidence artifact.

The official labs

Initial release: three AWS variants (SC-28, AC-3, CM-6), gate could report `PASS` having evaluated nothing.